COMP90050 ADBS

Week 7

Q1 ACID Isolation

1. Isolation property in ACID properties states that each transaction should run without interfering or being aware of other transactions in the system. In that case, we can run each transaction sequentially by locking the whole database for itself. Review why this may not be an ideal solution.

Not ideal because:

- Not benefiting from the potential use of idle resources
  - E.g. accessing disk while another transaction is using the CPU
  - concurrency can speed up things even under single CPU situations
- A long-running transaction can hold up every other transaction

We want execution outcome to be equivalent to a serial execution, but running in concurrency for efficiency

Q2 Isolation & Lock

2. Given two transactions, per operation of each transaction, we can use locks to make sure concurrent access is done properly to individual objects that are used in both transactions i.e., they are not accessed at the same time. This is after all what the operating systems do, e.g., lock a file while one program is accessing it so others cannot change it at the same time. Give two transactions showing that this is not enough to achieve the isolation property of transactions for RDBMSs.

I.e. locks can be requested and released for each read/write operation within the transaction.
Task: design 2 transactions and an execution order, that when adhere to the locking rules, would still violate isolation property (e.g. would give different outcome to serial execution)

This execution outcome is not equal to the outcome of a serial execution and does not obey the Isolation property

- Either T1 running first or T2 running first would give A=150 at the end

We need something more for proper concurrency control in DBMSs. (e.g. well formed and two-phased)

Answer (A is initially 0 on disk)	Transaction 1 at Process 1	Transaction 2 at Process 2
Operation 1	Lock(A)	
Operation 2	Read(A)	
Operation 3	Unlock(A)	
Operation 4	A = A +100	
Operation 5		Lock(A)
Operation 6		Read(A)
Operation 7		A = A + 50
Operation 8		Write(A)
Operation 9		Unlock(A)/Commit A=50
Operation 10	Lock(A)	
Operation 11	Write(A)	
Operation 12	Unlock(A)/Commit A=100	

Q3 Deadlock Probability

3. What is the probability that a deadlock situation occurs?

Assume:

- Number of transactions = n+1 $\approx n$ (if n is big)
- Each transaction access r locks exclusively
- Total number of records in the database = R

1. On average, each transaction hold approximately $\frac{r}{2}$ locks (min=0, max=r, average=$\frac{r}{2}$)
2. Average #locks taken by the other **n** transactions = $\frac{\boldsymbol{nr}}{2}$
3. P(transaction T waits for a lock) = P(request a lock on one of the $\frac{nr}{2}$ taken locks **out of R possible locks**) = $\frac{nr}{2\boldsymbol{R}}$
4. P(T **did not wait** for a lock) = $\mathbf{1} - \frac{nr}{2R}$
5. P(T did not wait for **any of the r locks** it needs) = $\left(1 - \frac{nr}{2R}\right)^{\boldsymbol{r}} \approx 1 - \left(\frac{nr^2}{2R}\right)$
6. P(T **waits** in its life time) = $\mathbf{1} - \left(1 - \left(\frac{nr^2}{2R}\right)\right) = \frac{nr^2}{2R}$
7. P(T waits for some $T1$ and $\boldsymbol{T1}$ **also waits for** $\boldsymbol{T}$) = $\frac{nr^2}{2R} \times \frac{\mathbf{1}}{\boldsymbol{n}} \cdot \frac{\boldsymbol{nr^2}}{\boldsymbol{2R}} = \frac{nr^4}{4R^2}$
   - $P(T1$ waits for $T) = \frac{1}{n} \times P(T1$ wait for someone)
   - i.e. P(deadlock by a particular transaction T with any other transaction)
8. P(deadlock by **any two transactions**) = $\mathbf{n} \cdot \frac{nr^4}{4R^2} = \frac{n^2r^4}{4R^2}$ (a very small probability in practice)

Q4 Execution Orders

4. Given the following two tasks where initial value of B is 50, and operations are labeled as Op1, Op2, etc, per task and we know that Task 1 has done its first operation already, please show all possible concurrent execution orders of the following tasks. Then state which orders make sense and which ones do not. If these were two transactions running concurrently this way, what property of transactions we would be violating with the invalid concurrent execution orders? The tasks are:

   - Task 1:
     - Op1) $B = B \times 2$
     - Op2) $B = B \times 2$
   - Task 2:
     - Op1) $B = B + 10$

Possible order:

1. Task 1 Op1 ($\times 2$), Task 1 Op2 ($\times 2$), Task 2 Op1 ($+10$)
2. Task 1 Op1($\times 2$), Task 2 Op1 ($+10$), Task 1 Op2 ($\times 2$)

Output:

$B = 50 \times 2 \times 2 + 10 = 210$

$B = (50 \times 2 + 10) \times 2 = 220$

Only 210 makes sense as it is equivalent to the outcome of serial execution of Task 1 then Task 2

- Serial execution of Task 2 then Task 1 would get B=240≠220 as well

Not adhering to the **Isolation property** of transactions

- the two transactions running in a peculiar way concurrently and it is the only way we can get 220 and hence the two transactions know about each other!